



MattRM2 VFX Build

Release 1.1.0

Github-> https://github.com/MattRM2/MattRM2_VFX_Build.git

PYSIDE6 · BRIDGE · PIPELINE TOOLS

Qt Tools — The Basics

The Qt mini-server — MattRM2 VFX Build

How to build your own tool windows (PySide6) wired to Blender.
Every function explained, plus three step-by-step tutorials — from reading settings to sending actions.

No prior coding knowledge required.

Created by Matthieu Barbié · 2026

v1.1

Unofficial build — not affiliated with or endorsed by the Blender Foundation.

0

Contents

1

Understanding the Qt mini-server

- 1.1 The idea in one picture
- 1.2 Why two separate programs?
- 1.3 The 4 pieces of the system
- 1.4 The shared language (the protocol)
- 1.5 The vocabulary to know

2

Function reference

- 2.1 The bridge's public API — what you use
- 2.2 The SDK — `run_control_panel` and its helpers
- 2.3 The bridge's internal plumbing
- 2.4 Auto-loading and the demo add-on

3

Tutorial 1 — “Hello World”

- 3.1 Before you start
- 3.2 Write your first tool
- 3.3 Launch it from the Python console
- 3.4 Test the two-way sync
- 3.5 What happened, step by step

4

Tutorial 2 — The “Output” panel

- 4.1 The goal
- 4.2 Understanding the setting “paths”
- 4.3 Write the full tool
- 4.4 Wrap it in a real add-on
- 4.5 Install, enable, use
- 4.6 Common pitfalls

5

Tutorial 3 — Sending instructions to Blender

- 5.1 The principle: remote calls
- 5.2 Step 1 — the action on the Blender side
- 5.3 Step 2 — the Qt window with a button
- 5.4 Step 3 — the add-on that ties it together
- 5.5 Step 4 — install and test
- 5.6 Going further

Who this guide is for

*This guide is deliberately very gradual. If a sentence feels obvious to you, great — skip to the next one. The goal is that **anyone**, even with no coding experience at all, can build a small tool window wired to Blender.*

Download the examples

The code of every tutorial is also available ready to use in **MattRM2_Qt_Examples.zip**, on the release page: github.com/MattRM2/MattRM2_VFX_Build/releases/latest. It holds the scripts and the two add-ons already packed for Install from Disk... — no need to copy code out of this PDF.

1

Understanding the Qt mini-server

The core ideas, explained simply

The **Qt mini-server** (its real name is the **MattRM2 Qt Bridge**) is there to show modern tool windows — settings panels, an asset browser, a render-farm submitter — that go **beyond** what Blender's native UI allows. These windows are written with **Qt** (the graphics library), via **PySide6** (Qt's Python version).

Good news: you have to write **almost no** code. A complete tool often fits in about twenty lines. All the hard work (creating the widgets, talking to Blender, keeping values in sync) is already done for you. Let's see how.

1.1 The idea in one picture

Picture **two programs** running side by side on your computer: Blender on one side, your tool window on the other. They talk to each other over a kind of local “phone line” (a *network socket* on your own machine).

PROCESS 1 — BLENDER

The **Bridge** (server, no UI).

Reads and writes Blender's settings and launches the tools.

←→
TCP socket
local · JSON

PROCESS 2 — YOUR TOOL

The **SDK** + your little script.

The PySide6 window: checkboxes, numbers, lists, paths...

When you move a setting in the window, it “phones” Blender: “change this setting”. And when a setting changes in Blender, Blender “calls back” the window so it updates. This is what we call **two-way sync**.

1.2 Why two separate programs?

We could have shown the Qt window *inside* Blender. But that caused a big problem: as soon as you moved Blender's main window, everything froze. The chosen solution is to run each tool in its **own program** (we call it a **separate process**):

- **Never freezes** — moving or resizing Blender's window never freezes the tool again (and vice-versa). Each one lives its own life.
- **Always on top** — tool windows stay in the foreground, so you never lose them behind Blender.
- **Safety** — if a tool crashes, Blender keeps running just fine.
- **Freedom** — the tool has its *own* Python: you can add external libraries to it without touching Blender.

The server is already there, silently

*The bridge (the server) is **loaded automatically** when Blender starts and has no UI of its own. It simply waits for someone to ask it to launch a tool. You don't have to enable anything for it.*

1.3 The 4 pieces of the system

The system is made of four files. Knowing each one's role helps you understand everything else:

Piece	Its role
1. The Bridge <code>mattrm2_qt_bridge</code>	The server , on the Blender side. No UI, loaded automatically. It launches the tools, reads/writes Blender's settings, and relays the messages. This is the one you call with a single line to open a tool.
2. The SDK <code>mattrm2_qt_sdk</code>	The "toolbox" on the window side. It builds the right checkbox / field for each setting all by itself, applies the dark theme, and handles the sync. Your script imports it and just hands it a list of settings.
3. Your tool <code>my_tool.py</code>	A tiny Python script: a list of groups and settings. This is the only thing you really write.
4. The launcher add-on (optional)	A small Blender add-on that adds a button (e.g. in the N-panel) to open your tool in one click. Optional: you can also launch the tool from the console or a custom menu.

1.4 The shared language (the protocol)

The two programs exchange small **messages** over the phone line. Each message is a piece of text in **JSON** format (a standard way of writing data) ending with a line break. You will **never** have to write these messages by hand — the SDK and the bridge do it for you — but knowing them helps you understand the "magic":

Message	What it means
Qt Blender : bind	"Here is my list of settings (their path, their label, their group). Build me the right form." — sent once, on connection.
Qt Blender : set	"The user just changed this setting; apply it in Blender."
Qt Blender : call	"Run this Blender function or operator, and send me back the result." (a remote call, known as RPC)

Message	What it means
Blender Qt : schema	"For each setting: its type (checkbox, number, list...), its bounds, its options." The SDK uses it to build the right fields.
Blender Qt : state	"Here are the current values of all the settings." — at the start, then every time a value changes in Blender.
Blender Qt : result	"Here is the result of your call." (the reply to an RPC call)

1.5 The vocabulary to know

Five words come up over and over. Learn them once and for all:

Word	Simple definition
RNA path	The address of a setting in Blender, written with dots. Example: scene.render.resolution_x = "the X resolution, in the render settings, of the scene". This is what you write for each field of your tool.
Property	A setting you can read and change (the resolution, a checkbox, a format...). Each property has a type .
Type	The nature of a property: BOOLEAN (checkbox), INT (whole number), FLOAT (decimal number), ENUM (a list of choices), STRING (text / file path). The SDK picks the right field based on the type.
Process	A running program. Blender is one process; each Qt tool is another, separate one.
Operator	A Blender action triggered by a button (e.g. "open the tool"). In code, its name looks like wm.my_tool .

2

Function reference

Every function in the code, explained

This section describes **all** the functions in the code, on **two levels**: first the ones you actually call (easy, worth knowing), then the internal “plumbing” (useful to understand, but that you will never have to touch).

2.1 The bridge's public API — what you use

To use it, you import the bridge at the top of an add-on or in the console:

```
import matrm2_qt_bridge as qt
```

Here are the eight public functions (the “API”) and their role:

Function	Role
<code>qt.available()</code>	Returns True if the bridge is ready. Used to check, before launching a tool, that the server is indeed there.
<code>qt.launch_tool(path)</code>	The key function. Launches the given tool script in its own process, opens the phone line, and returns True once the process has started. This is what a button will call.
<code>qt.register_tools_dir(folder)</code>	Declares a folder full of tool scripts, to list them automatically (handy for a “my tools” menu).
<code>qt.unregister_tools_dir(folder)</code>	Removes a previously declared folder.
<code>qt.list_tools()</code>	Returns the list (label, path) of the tools found in the declared folders (the .py files, ignoring those starting with “_”).
<code>qt.register_handler(name, fn)</code>	Wires one of <i>your</i> Python functions on the Blender side, that the tool can call remotely by its name (the RPC mechanism). For advanced business logic.
<code>qt.unregister_handler(name)</code>	Unwires a previously registered handler.
<code>qt.sdk_dir()</code>	Returns the bundled SDK folder. The bridge adds it to the PYTHONPATH of the tools it launches all by itself — you normally don't need it.

The two you will actually use

```
qt.launch_tool(script_path) -> bool
```

Give it the full path of your tool script. The bridge starts a new Python process (Blender's), makes the SDK available to it, opens a local socket, and passes the “line” number (port) to the tool. Returns True once the process has started — it does not check that your script runs: if the path is wrong or the script has an error, it still returns True and no window opens (the error is in the system console). The same tool can be launched several times — each call opens a new window.

```
qt.register_handler(name, fn)
```

For advanced tools. **fn** is a function that receives a dictionary of arguments and returns a simple value (JSON-serializable). The tool then calls it by its **name**. Example use: trigger custom business logic on the Blender side (a pipeline tool that registers its own actions). For plain settings panels, you won't need it.

2.2 The SDK — run_control_panel and its helpers

The SDK runs in the tool's process (not in Blender). It contains one main function and a few internal helpers.

The only function your script calls

```
run_control_panel(title, layout)
```

A tool's **entry point**. Give it a title and a **layout**: a list of groups, each group containing (label, RNA path) pairs. The function reads the line number (–port), creates the Qt application, opens the window and wires it to Blender. From there, everything is automatic. **This, on its own, is a whole tool.**

```
from matrm2_qt_sdk import run_control_panel

run_control_panel("My Tool", layout=[
    ("Group", [
        ("Label", "scene.render.some_setting"),
    ]),
])
```

The SDK's internal helpers (to understand, not to call)

Function	Role
<code>_make_card(title)</code>	Builds a visual “card” (the rounded dark frame) with a group title and an empty form, ready to receive the fields.
<code>make_field(field, on_set)</code>	The heart of the SDK: from a setting's description (its type), it creates the right widget — checkbox, number spinner, drop-down list, or a text field with a “...” button to browse files. It notifies the tool when the user changes the value.
<code>ControlPanel(...)</code>	The window class. It opens the line to Blender, sends the “bind”, receives the schema and the state, builds the cards and keeps everything in sync.
<code>ControlPanel._on_connect()</code>	The moment the line opens: sends the “bind” message (the list of all your settings) to Blender.
<code>ControlPanel._on_set(path, value)</code>	When you change a field: sends a “set” message to Blender to apply the new value.
<code>ControlPanel._on_read()</code>	Reads the messages coming from Blender: “schema” (builds the UI) and “state” (updates the displayed values).
<code>ControlPanel._build(groups)</code>	Builds the UI once, from the received schema: one card per group, one field per setting.
<code>ControlPanel._apply_state(values)</code>	Copies Blender's values into the fields, without clobbering the one you are currently editing (so it doesn't cut you off mid-typing).

The key takeaway

*Remember just **run_control_panel**. All the rest of the SDK works for you behind the scenes; it is listed here for background knowledge only.*

2.3 The bridge's internal plumbing

These functions are the “engine” on the Blender side. You will never call them, but here is their role, grouped by mission. **Important detail:** all of this runs on Blender’s main thread, via a small timer that checks the line ~20 times per second without ever blocking – that’s what makes the whole thing safe and freeze-proof.

Group A — Read and write Blender's settings

Function	Role
<code>_resolve(path)</code>	Turns a path like “scene.render.ffmpeg.codec” into the concrete object + the name of the last setting. It's the one that knows how to “walk” along the dots.
<code>_get(path)</code>	Reads the current value of a setting.
<code>_set(path, value)</code>	Writes a new value, converting it to the right type (integer, decimal, true/false) according to the property.
<code>_field_schema(path, label, group)</code>	Inspects a setting and describes its type, its bounds, its options. This is the “schema” the SDK uses to build the right field.

Group B — Launch and run a tool

Function	Role
<code>_python_exe()</code>	Finds the Python interpreter <i>bundled</i> inside Blender (not blender.exe) to start the tool's process.
<code>_call_target(target, args)</code>	Runs either one of your registered handlers, or a Blender operator (bpy.ops...), and returns the result. Powers the remote-call mechanism (RPC).
<code>class _Session</code>	Represents one running tool: its socket, its process, its buffers, the list of tracked settings. One session per open window.
<code>_Session.start()</code>	Opens a listening socket on a free port, then launches the tool's process, passing it that port and access to the SDK.
<code>_Session.tick()</code>	A session's heartbeat: accepts the connection, reads incoming messages, and periodically (every ~0.3 s, only if it changed) pushes Blender's state to the tool.
<code>_Session._handle(line)</code>	Handles a message received from the tool: “bind” (returns schema + state), “set” (applies a setting), “call” (runs and replies).
<code>_Session._send(obj)</code>	Sends a JSON message to the tool over the socket.
<code>_Session.dead()</code>	Tells whether the tool's process has stopped (window closed).
<code>_Session.stop()</code>	Cleanly closes the socket and stops the tool's process.

Group C — The timer and the lifecycle

Function	Role
<code>_ensure_timer()</code>	Starts the global timer once (as soon as at least one tool is running).
<code>_global_tick()</code>	The conductor, ~20 times/second: “ticks” each session, removes the dead ones, and stops the timer when no tool is open anymore.
<code>register()</code>	Called when the bridge add-on is enabled. Since the bridge has no UI, it just prints a small “ready” message.
<code>unregister()</code>	Called on disable: stops all open sessions and removes the timer.

2.4 Auto-loading and the demo add-on

Two last files complete the system. The first **turns on** the bridge by itself at startup; the second is the reference **example** you'll base your own launcher add-ons on.

The auto-loader — `scripts/startup/mattrm2_autoload.py`

Function	Role
<code>register()</code>	Called by Blender at launch. It schedules the bridge to be enabled just after startup (once the add-on system is ready).
<code>_enable()</code>	Actually enables the bridge (and only it; the Deep EXR demo tool stays manual). Runs only once.
<code>unregister()</code>	Does nothing — there is nothing to undo for a simple loader.

The demo add-on — `mattrm2_deepexr` (your template to copy)

This is the exact pattern to follow to make a button that opens a tool. It adds a “MattRM2 Qt” tab in the 3D view N-panel with an “Open Deep EXR Panel” button.

Function	Role
<code>_get_bridge()</code>	Imports the bridge on demand and returns it if available (otherwise None). Lets you warn the user cleanly if the server is missing.
<code>WM_OT_..._deepexr</code> (Operator)	The action behind the button. Its <code>execute()</code> method gets the bridge and calls <code>qt.launch_tool()</code> on the tool's script.
<code>VIEW3D_PT_...</code> (Panel)	The panel in the N-panel. Its <code>draw()</code> method draws the button that triggers the operator.
<code>register()</code> / <code>unregister()</code>	Register and remove the operator and the panel with Blender (the ritual of every add-on).

3

Tutorial 1 — “Hello World”

Your very first tool window, step by step

We're going to create the smallest possible window: two settings wired to Blender. You'll see the **sync magic** live. Allow less than five minutes, with no experience required.

3.1 Before you start

All you need is:

- The **MattRM2 VFX Build** open (the bridge loads by itself — nothing to enable).
- Any **text editor** (Windows Notepad is perfectly fine).
- Knowing where you save your files (e.g. your Desktop).

Nothing to install

You don't need to install PySide6 or anything else: it's all already bundled in the build.

3.2 Write your first tool

1 Open your text editor and create a new file.

2 Copy in **exactly** the lines below.

```
from mattrm2_qt_sdk import run_control_panel

run_control_panel(
    "My First Tool",
    layout=[
        ("Image", [
            ("Width (px)", "scene.render.resolution_x"),
            ("Transparent film", "scene.render.film_transparent"),
        ]),
    ],
)
```

3 Save the file as **my_first_tool.py** (double-check the **.py** extension, not **.txt**) — for example on your Desktop.

Let's break these lines down:

- The first line **imports** the SDK's magic function.
- **“My First Tool”** is the window title.

- **layout** is the layout: a list of **groups**. Here a single group, “Image”.
- Each line of a group is a (**label, RNA path**) pair: the text shown, then the address of the setting in Blender.
- **scene.render.resolution_x** is a number you get a spinner. **scene.render.film_transparent** is a true/false you get a checkbox. You do **not** have to specify it: the SDK figures it out on its own.

3.3 Launch it from the Python console

No add-on needed for this first try: we launch the tool directly from Blender's **Python Console**.

1

In Blender, open a **Python Console** editor (change an area's type via the icon at the top left, *Scripting* category; or use the **Scripting** workspace at the top of the window).

2

Type the two lines below (adjust the path to *your* file), pressing Enter after each line:

```
import matrm2_qt_bridge as qt
qt.launch_tool(r"C:\Users\YourName\Desktop\my_first_tool.py")
```

A small dark window “**My First Tool**” opens, with a “Width (px)” spinner and a “Transparent film” checkbox.

Mind the Windows paths

On Windows, be sure to put an **r** right before the quotes: **r"C:\...**". That “**r**” keeps the backslashes (\) from causing trouble. Alternatively: replace each \ with a / (e.g. "C:/Users/YourName/Desktop/my_first_tool.py").

3.4 Test the two-way sync

This is the satisfying part. Try the two tests below:

A

In the tool window, **increase** “Width (px)”. Watch Blender: in **Properties > Output > Format**, Resolution X moves at the same time.

B

Now the other way around: change Resolution X **in Blender**. The value updates **by itself** in the tool window.

C

Tick “Transparent film” in the tool, then check in **Properties > Render > Film**: the “Transparent” checkbox follows.

Well done!

You just built an external tool, in a real PySide6 window, synced in real time with Blender — in **10 lines**. That's the whole principle.

3.5 What happened, step by step

To tie what you did back to the “functions” section:

- **qt.launch_tool(...)** started a new Python process and opened your script in it, with the SDK available and a phone line (socket) already connected.
- **run_control_panel(...)** created the Qt window and sent the **bind** message (“here are my settings”).
- The bridge replied with **schema** (the types) then **state** (the values), and the SDK built the spinner and the checkbox.
- When you move a field **set** message the bridge writes into Blender. When Blender changes **state** message the window updates. The loop is complete.

4

Tutorial 2 — The “Output” panel

A window that drives the image output settings

Let's move to a tool that's useful day to day: a window that gathers the main settings of Blender's **Output** panel — resolution, file format, color mode, output path, etc. And we'll wrap it in a real **add-on** with a button.

4.1 The goal

The result: a floating window, always on top of Blender, organized into three groups (Dimensions, Output File, Options). Handy to tweak your render settings without digging through the tabs, or to keep those settings in plain sight on a second monitor.

4.2 Understanding the setting “paths”

The whole trick is finding the right **RNA path** for each setting. Good news: Blender gives it to you! First turn on **Edit > Preferences > Interface > Display > Python Tooltips** (it is off by default). Then hover any setting with the mouse: the tooltip shows its Python path at the very bottom. You can also **right-click > Copy Data Path** (Ctrl+Shift+C) and paste it.

Always start with scene.

*The bridge reads paths starting from the **current scene**, so every path must begin with **scene**. — for example **scene.render.resolution_x**. **Copy Data Path** gives **render.resolution_x**: just add **scene.** in front. Avoid **Copy Full Data Path**: it gives **bpy.data.scenes["Scene"].render.resolution_x**, which the bridge cannot read.*

Output settings almost all live under **scene.render**, and the image-file ones under **scene.render.image_settings**. Here are the ones we'll use:

RNA path	What it does
<code>scene.render.resolution_x / _y</code>	Image width and height, in pixels (numbers).
<code>scene.render.resolution_percentage</code>	Render scale, in % (a number, e.g. 50 = half).
<code>scene.render.fps</code>	Frames per second (for animation).
<code>scene.render.filepath</code>	The output folder / path. It's a path: the SDK will add a “...” browse button all by itself.
<code>scene.render.image_settings.file_format</code>	The file format (PNG, JPEG, OpenEXR...) — a list of choices.
<code>scene.render.image_settings.color_mode</code>	Grayscale, RGB or RGBA — a list.
<code>scene.render.image_settings.color_depth</code>	Color depth (8, 16, 32 bits...) — a list.

RNA path	What it does
<code>scene.render.use_overwrite</code>	Overwrite existing files (true/false).
<code>scene.render.use_file_extension</code>	Add the extension automatically (true/false).
<code>scene.render.use_placeholder</code>	Create placeholder files before rendering (true/false).

4.3 Write the full tool

We use exactly the same recipe as in tutorial 1, but with more lines and **three** groups. Create a file **output_tool.py**:

```
from mattrm2_qt_sdk import run_control_panel

run_control_panel(
    "Image Output",
    layout=[
        ("Dimensions", [
            ("Resolution X", "scene.render.resolution_x"),
            ("Resolution Y", "scene.render.resolution_y"),
            ("Scale (%)", "scene.render.resolution_percentage"),
            ("Frame rate", "scene.render.fps"),
        ]),
        ("Output File", [
            ("Output path", "scene.render.filepath"),
            ("Format", "scene.render.image_settings.file_format"),
            ("Color mode", "scene.render.image_settings.color_mode"),
            ("Color depth", "scene.render.image_settings.color_depth"),
        ]),
        ("Options", [
            ("Overwrite", "scene.render.use_overwrite"),
            ("File extension", "scene.render.use_file_extension"),
            ("Placeholders", "scene.render.use_placeholder"),
        ]),
    ],
)
```

You can already test it like in tutorial 1, from the console: `qt.launch_tool(r"...\\output_tool.py")`. The window shows three cards, with a field suited to each setting, and a "..." button next to the output path.

4.4 Wrap it in a real add-on

Launching from the console is fine for testing. For daily use, we want a **button**. We create a small add-on, modeled on the Deep EXR demo. Put this code in an `__init__.py` file, **in the same folder** as `output_tool.py`:

```
bl_info = {
    "name": "My Output Panel (Qt)",
```

```

    "author": "Your Name",
    "version": (1, 0, 0),
    "blender": (5, 2, 0),
    "location": "View3D > N-panel > My Tools",
    "category": "Render",
}

import os, bpy

_THIS_DIR = os.path.dirname(os.path.abspath(__file__))
_QT_SCRIPT = os.path.join(_THIS_DIR, "output_tool.py")

def _get_bridge():
    try:
        import mattrm2_qt_bridge as bridge
        return bridge if bridge.available() else None
    except Exception:
        return None

class WM_OT_my_output(bpy.types.Operator):
    bl_idname = "wm.my_output_panel"
    bl_label = "Open Output Panel"

    def execute(self, context):
        bridge = _get_bridge()
        if bridge is None:
            self.report({'ERROR'}, "Qt bridge not available.")
            return {'CANCELLED'}
        bridge.launch_tool(_QT_SCRIPT)
        return {'FINISHED'}

class VIEW3D_PT_my_output(bpy.types.Panel):
    bl_label = "My Tools"
    bl_space_type = "VIEW_3D"
    bl_region_type = "UI"
    bl_category = "My Tools"

    def draw(self, context):
        self.layout.operator("wm.my_output_panel", icon="WINDOW")

_classes = (WM_OT_my_output, VIEW3D_PT_my_output)

def register():
    for c in _classes:
        bpy.utils.register_class(c)

def unregister():
    for c in reversed(_classes):
        bpy.utils.unregister_class(c)

```

What does this code do? In plain words:

- **bl_info** is the add-on's ID card (name, version, where to find it).
- **_get_bridge()** gets the server, or warns gently if it's missing.
- **WM_OT_my_output** is the action: when clicked, it calls **bridge.launch_tool()** on your **output_tool.py**.

- **VIEW3D_PT_my_output** draws the button in a “My Tools” N-panel tab.
- **register / unregister** wire and unwire everything in Blender.

4.5 Install, enable, use

So you have a folder with **two** files:

```
my_output_panel/
  __init__.py      (the add-on, the button)
  output_tool.py   (the Qt window)
```

- 1 Compress the **folder** my_output_panel into a **.zip** file (right-click > Compress / Send to > Compressed folder). Or skip this step: **my_output_panel.zip** is ready in MattRM2_Qt_Examples.zip.
- 2 In Blender: **Edit > Preferences > Add-ons > Install from Disk...**, then select your .zip.
- 3 Tick the box to **enable** the “My Output Panel (Qt)” add-on.
- 4 In the 3D view, open the **N-panel** (the **N** key), go to the **My Tools** tab, and click **Open Output Panel**.

And there's your home-made tool

The window opens, already wired to Blender. Change the format or the resolution in the tool: Blender's Output panel follows live, and vice-versa.

4.6 Common pitfalls

Symptom	Fix
The window doesn't open	Check the script path (typos), and that output_tool.py is indeed next to the __init__.py. Open Blender's system console (Window > Toggle System Console) to read any error message.
“Qt bridge not available”	The server isn't loaded. It normally is, by itself; if in doubt, check in the Preferences that the MattRM2 — Qt Bridge add-on is present.
A list's options seem frozen	The schema (the choices of a list like “Color depth”) is computed once, at opening. If you change the Format in Blender, close and reopen the tool to refresh the corresponding options.
The field I'm editing “jumps”	That's normal while typing: the sync avoids clobbering the active field. The value settles as soon as you confirm (Enter) or click elsewhere.

Symptom	Fix
Wrong .py saved as .txt	In Notepad, choose “All files” and type the full name with .py, otherwise Windows adds an invisible .txt.

5

Tutorial 3 — Sending instructions to Blender

Trigger an action in one click: build a complete Suzanne

So far, our tools **read and wrote settings**. We can also ask Blender to **run an action**: create an object, add a modifier, assign a material... We're going to build, in a single click, a **Suzanne** that's subdivided, smooth-shaded and painted in the Matt Theme orange.

5.1 The principle: remote calls

To trigger an action, we use the **remote call** (RPC in English), carried by the **call** message. Two new things compared to the previous tutorials:

- **On the Blender side** — we write a function (a "handler") that does the work, and we register it: `qt.register_handler("build_suzanne", build_suzanne)`.
- **On the window side** — when the button is clicked, we send a **call** message naming that handler. Blender runs it on its main thread (so no risk), then replies with a **result** message.

Don't forget

*The handler must be **registered** before the call. Otherwise Blender thinks "build_suzanne" is an operator, can't find it, and returns an error.*

5.2 Step 1 — the action on the Blender side

Here is the function that performs the four requested operations. It receives a dictionary of arguments (unused here) and returns the name of the created object:

```
def build_suzanne(args):
    # 1. Create Suzanne
    bpy.ops.mesh.primitive_monkey_add()
    obj = bpy.context.active_object
    obj.name = "Suzanne"

    # 2. Subdivision Surface modifier, level 2
    mod = obj.modifiers.new(name="Subdivision", type="SUBSURF")
    mod.levels = 2 # in the 3D view
    mod.render_levels = 2 # at render

    # 3. Shade smooth: mark every face as smooth
    for poly in obj.data.polygons:
        poly.use_smooth = True

    # 4. Orange Principled BSDF material (#FB923C)
    mat = bpy.data.materials.new(name="Matt Orange")
```

```

bsdf = mat.node_tree.nodes.get("Principled BSDF")
if bsdf:
    bsdf.inputs["Base Color"].default_value = (0.965, 0.287, 0.045, 1.0)
obj.data.materials.clear()
obj.data.materials.append(mat)

return obj.name

```

Step by step:

- **1. primitive_monkey_add()** adds the monkey head; we grab the active object (the one we just created) and rename it "Suzanne".
- **2. modifiers.new(...)** adds a Subdivision Surface modifier: **levels** sets the smoothness in the view, **render_levels** at render.
- **3.** "Shade Smooth" comes down to marking every face as smooth (**use_smooth = True**). We do it via the data: that works in every case.
- **4.** We create a material (it comes with its nodes already), grab the "Principled BSDF" node, set its **Base Color**, then assign it to Suzanne.

The color, in linear

Why (0.965, 0.287, 0.045) rather than (0.98, 0.57, 0.23)? Because Blender works in **linear** colors. This triplet is the Matt Theme orange #FB923C converted to linear — that's the one that will actually look orange on screen.

Safe

This handler runs on Blender's **main thread** (the bridge calls it from its timer), so all the bpy commands are safe there.

5.3 Step 2 — the Qt window with a button

For an **action button**, **run_control_panel** isn't the right fit: it only handles settings. So we write a small window ourselves. It's a bit more code, but the pattern stays simple: connect to the bridge, and on click send a **call** message. Create the file **actions_tool.py**:

```

import sys, json, argparse
from PySide6 import QtWidgets, QtCore, QtNetwork

class Tool(QtWidgets.QWidget):
    def __init__(self, port):
        super().__init__()
        self.setWindowTitle("Suzanne Express")
        self.setWindowFlag(QtCore.Qt.WindowStaysOnTopHint, True)
        self._buf = b''
        root = QtWidgets.QVBoxLayout(self)
        self.status = QtWidgets.QLabel("Connecting to Blender...")
        self.btn = QtWidgets.QPushButton("Build Suzanne")
        self.btn.setEnabled(False)
        self.btn.clicked.connect(self._build)
        root.addWidget(self.status)

```

```

        root.addWidget(self.btn)
        self.sock = QtNetwork.QTcpSocket(self)
        self.sock.readyRead.connect(self._on_read)
        self.sock.connected.connect(self._on_connect)
        self.sock.disconnected.connect(self.close)
        self.sock.connectToHost("127.0.0.1", int(port))

    def _on_connect(self):
        self.status.setText("Connected to Blender - ready")
        self.btn.setEnabled(True)

    def _build(self):
        self.status.setText("Building...")
        msg = {"op": "call", "target": "build_suzanne", "args": {}, "id": 1}
        self.sock.write((json.dumps(msg) + "\n").encode("utf-8"))

    def _on_read(self):
        self._buf += bytes(self.sock.readAll())
        while b"\n" in self._buf:
            line, _, self._buf = self._buf.partition(b"\n")
            if not line.strip():
                continue
            msg = json.loads(line.decode("utf-8"))
            if msg.get("op") == "result":
                self.status.setText("Done: " + str(msg.get("value")))

ap = argparse.ArgumentParser()
ap.add_argument("--port", type=int, required=True)
args, _ = ap.parse_known_args()
app = QtWidgets.QApplication(sys.argv)
win = Tool(args.port)
win.show()
sys.exit(app.exec())

```

- We **connect** to the bridge (**connectToHost**) with the line number received as an argument (`--port`).
- On click, **_build** sends a **call** message naming the “build_suzanne” handler.
- **_on_read** reads the **result** reply and shows the name of the created object.
- The window stays on top of Blender and closes if Blender disconnects.

The theme, optional

Without a stylesheet, the window has the default Qt look. For the Matt dark theme, add a `self.setStyleSheet("...")` like in the SDK.

5.4 Step 3 — the add-on that ties it together

The add-on does two things: it **registers the handler** (so **call** can find it) and it adds a **button** to open the window. Here is the `__init__.py` file; it looks like the one from tutorial 2, plus the handler registration:

```
bl_info = {
```

```

    "name": "Suzanne Express (Qt)",
    "author": "Your Name",
    "version": (1, 0, 0),
    "blender": (5, 2, 0),
    "category": "Object",
}

import os, bpy

_THIS_DIR = os.path.dirname(os.path.abspath(__file__))
_QT_SCRIPT = os.path.join(_THIS_DIR, "actions_tool.py")

# <<< Paste the build_suzanne function from step 1 here >>>

def _get_bridge():
    try:
        import matrm2_qt_bridge as bridge
        return bridge if bridge.available() else None
    except Exception:
        return None

class WM_OT_suzanne(bpy.types.Operator):
    bl_idname = "wm.suzanne_express"
    bl_label = "Open Suzanne Express"
    def execute(self, context):
        bridge = _get_bridge()
        if bridge is None:
            self.report({'ERROR'}, "Qt bridge not available.")
            return {'CANCELLED'}
        bridge.launch_tool(_QT_SCRIPT)
        return {'FINISHED'}

class VIEW3D_PT_suzanne(bpy.types.Panel):
    bl_label = "My Tools"
    bl_space_type = "VIEW_3D"
    bl_region_type = "UI"
    bl_category = "My Tools"
    def draw(self, context):
        self.layout.operator("wm.suzanne_express", icon="MONKEY")

_classes = (WM_OT_suzanne, VIEW3D_PT_suzanne)

```

And the part that **wires everything** into Blender (note the register_handler):

```

def register():
    for c in _classes:
        bpy.utils.register_class(c)
    bridge = _get_bridge()
    if bridge:
        bridge.register_handler("build_suzanne", build_suzanne)

def unregister():
    bridge = _get_bridge()
    if bridge:

```

```
bridge.unregister_handler("build_suzanne")
for c in reversed(_classes):
    bpy.utils.unregister_class(c)
```

The piece to paste

Don't forget to **paste the build_suzanne function** (step 1) at the spot marked by the comment, otherwise `register_handler` won't have anything to register.

5.5 Step 4 — install and test

You have a folder with two files:

```
suzanne_express/
__init__.py      (the add-on: the button + the action)
actions_tool.py  (the Qt window with the button)
```

1

Compress the **folder** `suzanne_express` into a **.zip** file. Or skip this step: **suzanne_express.zip** is ready in `MattRM2_Qt_Examples.zip`.

2

In Blender: **Edit > Preferences > Add-ons > Install from Disk...**, select the **.zip**, then **enable** "Suzanne Express (Qt)".

3

In the 3D view, open the **N-panel** (the **N** key), the **My Tools** tab, and click **Open Suzanne Express**.

4

In the window, click **Build Suzanne**. A subdivided, smooth-shaded, orange Suzanne appears instantly in the scene.

Mission accomplished

You just drove Blender remotely: a click in an external window triggered a whole sequence of operations in the scene. That's the foundation of every pipeline tool.

5.6 Going further

- **Pass arguments** — fill **args** (e.g. `{"name": "Hero"}`) on the window side, and read them in the handler with `args.get("name")`.
- **Call an operator directly** — if **target** isn't a registered handler, the bridge tries a Blender operator. So **target = "object.shade_smooth"** runs `bpy.ops.object.shade_smooth()`, with no handler to write.
- **Several buttons** — add more buttons and more handlers in the same window to build a real action panel.
- **Combine both worlds** — a `run_control_panel` tool for the settings, another for the actions: it's up to you to assemble your toolbox.

In summary

You now know how to do it all: read and write settings (tutorials 1 and 2) and trigger actions in Blender (tutorial 3). From there, change the list of RNA paths or the content of your handlers to automate any task. Have fun! — Matthieu Barbié

License & trademark

MattRM2 VFX Build is an independent build based on Blender 5.2.0, distributed under the GNU GPL v3. It also bundles Qt for Python (PySide6) and Qt under the GNU LGPL v3 (dynamically linked); the license texts are in the distribution's licenses/ folder. Not created, sponsored, or endorsed by the Blender Foundation. "Blender" is a trademark of the Blender Foundation. — blender.org